MyHarmony's data model

What the platform behind a Harmony remote actually stores: its entities, their fields, and how they refer to each other. Recovered on 30 August 2026 and checked against replies the service itself returned. `docs/myharmony/model.json` is the data, this is the reading, and section 218 is the argument. Two more views are generated from that data by `tools/myharmony_model.py` and never edited by hand: `docs/myharmony/core-model.mmd` draws the entities an account holds, and `docs/myharmony/entities.md` indexes all 470 service contracts by area. `make myharmony-model` checks they still agree with the model.

Consult this before designing anything about devices, activities or remotes, and before naming a field. It is a labelled view of bytes this project already reads: where `docs/config-format.md` says what a byte is, this says what the vendor **called** it and what else travelled beside it. Sections 159 to 171 derived an infrared rhythm table the hard way, and the vocabulary for the same thing is in here.

Why it can be trusted, and where it stops

The service is alive. `svcs.myharmony.com` answers, section 56. What is discontinued is the classic generation's service, so `SERVER-DEPENDENCY.md`'s "the domain model existed on the server only" is about a different, older client.

Three independent sources, which is what makes this a measurement rather than a reading:

- the **schema**, from the client's generated service proxy. A data contract is not the client's own design: it is the wire schema the **server** declared, which is why every type states the server side namespace it came from;
- **instances**, in the replies this project has captured from five accounts;
- and **the live service**, which can be asked about anything the other two do not exercise.

Against the captures, **nothing in the schema is missing from live data**. The captures come from this project's own test accounts on dates recorded in `reference/lab-register.md`, and those accounts change, so the table is a statement about what was captured. It is evidence that the schema is faithful, which is what it is for, and not a description of an account:

ENTITY	FIELDS IN THE SCHEMA	PRESENT IN A LIVE REPLY	EXTRA FIELDS THE REPLY CARRIED
Account	21	21	none
Activity	25	25	1
Device	32	32	10
Remote	32	32	3
Product	26	25	1

The service is ahead of this client build, which is the finding in the other direction: it returns fields the compiled proxy has never heard of. So a field absent here is not a field the platform lacks, and this model is a floor rather than a ceiling. The live extras are named in section 218.

What it is not. It is not the configuration compiler and does not become one. It says what the platform holds, not how a configuration is built from it.

The shape

1352 types, of which **470 are service contracts**, with **366 references between them** over 28 areas, and **1291 enum values**. The most connected types are identifier wrappers carrying no fields at all, so the useful shape is the one below.

A household has zero or more remotes, and each one comes with its own account record. That is the sentence to keep, and it took two corrections to reach: see below.

A `Household` holds `Account` records. Each account record carries **one** `Remote`, plus that remote's `Device` records and `Activity` records. So a household is the person, and an account is not a person and not a home: it is the container for a single remote and everything that remote controls. A `Remote` itself carries only what is true of the physical unit, its serial numbers, its skin, its firmware identifiers and its sync dates; a `Device` carries a manufacturer, a model, a category and a pointer into the catalogue.

The schema types all of these as lists, and the service is stricter than the schema. `Household` declares an `Accounts` list and a `Remotes` list, and `Account` declares a `Remotes` list of its own, so nothing in the schema bounds how many of anything there may be. Two things narrow it:

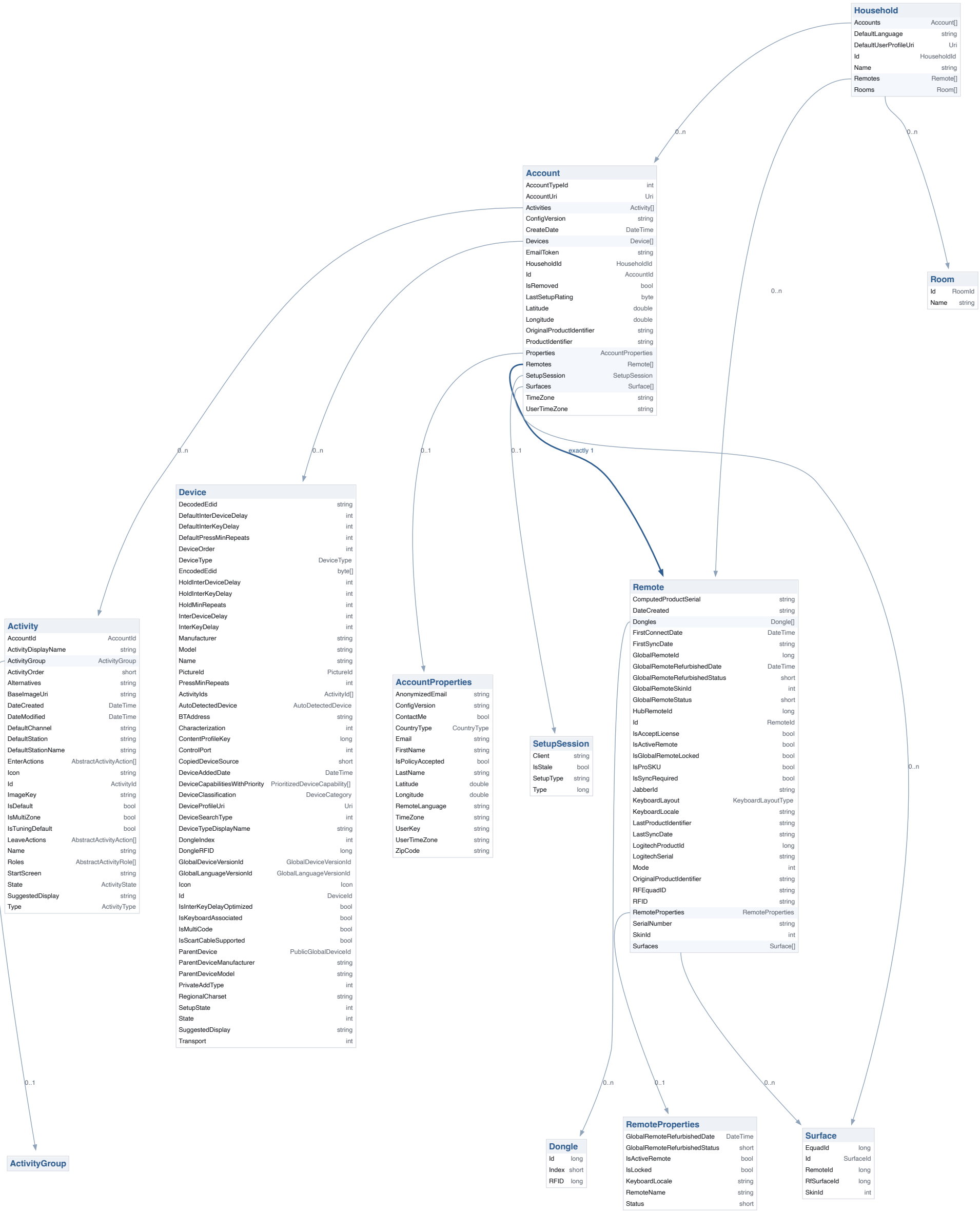
- `Household.Remotes` comes back **null** in every capture here, on both logins, while the account records carry a remote each. So the route to a remote is through the accounts, not through that field;
- an account record takes **one** remote, and that is the service's own rule rather than a count of ours: `UserAccountDirector/AddRemoteToAccount` **refuses** a record that already has one, with `ErrorCode 5`, measured on 13 August 2026 and written up in `docs/host-client.md`. The captures agree, 21 account records with exactly one remote each and 3 to 10 devices, but the refusal is the stronger statement and it was already in this repository.

So the product's shape and the schema's are the **same** shape. You own remotes, a remote has devices, and "the devices on an account" names exactly the devices of that account's one remote.

This paragraph was wrong twice on 30 August 2026, and both are worth keeping. First it read the field layout, saw `Remotes` and `Devices` as two lists side by side on `Account` and `Remote` carrying no device list, and concluded that the schema pools devices at account level in a shape the product does not present. The field layout is real and the conclusion does not follow: with one remote per account those two lists are not siblings competing to own the devices, they are a remote and its devices in the only container either has. Caught by Danny asking whether a remote in an account is therefore called a device, which the wording invited and nothing in the schema supports.

Then the replacement led with the wrong entity. It made **the account** the headline, when what a reader needs is that **a household has zero or more remotes**, the account layer being the mechanism by which it holds them rather than the shape itself. Caught by Danny reading the diagram.

Neither correction needed a new measurement, and that is the part to carry: `docs/host-client.md` has said since 13 August 2026 that a household holds one record per remote, that devices and activities hang off the account record rather than the household, and that the service refuses a second remote on a record. All three were re-derived here from captures instead of read from the document next door.



The entity cluster an account holds, generated from docs/myharmony/model.json. A heavy edge is a cardinality measured from captures rather than read from the schema.

Account

FIELD	TYPE
`AccountTypeId`	int
`AccountUri`	Uri
`Activities`	Activity list
`ConfigVersion`	string
`CreateDate`	DateTime
`Devices`	Device list
`EmailToken`	string
`HouseholdId`	HouseholdId
`Id`	AccountId
`IsRemoved`	bool
`LastSetupRating`	byte
`Latitude`	double
`Longitude`	double
`OriginalProductIdentifier`	string
`ProductIdentifier`	string
`Properties`	AccountProperties
`Remotes`	Remote list
`SetupSession`	SetupSession
`Surfaces`	Surface list
`TimeZone`	string
`UserTimeZone`	string

Remote

FIELD	TYPE
`ComputedProductSerial`	string
`DateCreated`	string
`Dongles`	Dongle list
`FirstConnectDate`	DateTime
`FirstSyncDate`	string
`GlobalRemoteId`	long
`GlobalRemoteRefurbishedDate`	DateTime
`GlobalRemoteRefurbishedStatus`	short
`GlobalRemoteSkinId`	int
`GlobalRemoteStatus`	short
`HubRemoteId`	long
`Id`	RemoteId
`IsAcceptLicense`	bool
`IsActiveRemote`	bool
`IsGlobalRemoteLocked`	bool
`IsProSKU`	bool
`IsSyncRequired`	bool
`JabberId`	string
`KeyboardLayout`	KeyboardLayoutType
`KeyboardLocale`	string
`LastProductIdentifier`	string
`LastSyncDate`	string
`LogitechProductId`	long
`LogitechSerial`	string
`Mode`	int
`OriginalProductIdentifier`	string
`RFEquadID`	string
`RFID`	string
`RemoteProperties`	RemoteProperties
`SerialNumber`	string
`SkinId`	int
`Surfaces`	Surface list

Device

FIELD	TYPE
`DecodedEdid`	string
`DefaultInterDeviceDelay`	int
`DefaultInterKeyDelay`	int
`DefaultPressMinRepeats`	int
`DeviceOrder`	int
`DeviceType`	DeviceType
`EncodedEdid`	byte list
`HoldInterDeviceDelay`	int
`HoldInterKeyDelay`	int
`HoldMinRepeats`	int
`InterDeviceDelay`	int
`InterKeyDelay`	int
`Manufacturer`	string
`Model`	string
`Name`	string
`PictureId`	PictureId
`PressMinRepeats`	int
`ActivityIds`	ActivityId list
`AutoDetectedDevice`	AutoDetectedDevice
`BTAddress`	string
`Characterization`	int
`ContentProfileKey`	long
`ControlPort`	int
`CopiedDeviceSource`	short
`DeviceAddedDate`	DateTime
`DeviceCapabilitiesWithPriority`	PrioritizedDeviceCapability list
`DeviceClassification`	DeviceCategory
`DeviceProfileUri`	Uri
`DeviceSearchType`	int
`DeviceTypeDisplayName`	string
`DongleIndex`	int
`DongleRFID`	long
`GlobalDeviceVersionId`	GlobalDeviceVersionId
`GlobalLanguageVersionId`	GlobalLanguageVersionId
`Icon`	Icon
`Id`	DeviceId
`IsInterKeyDelayOptimized`	bool
`IsKeyboardAssociated`	bool
`IsMultiCode`	bool
`IsScartCableSupported`	bool
`ParentDevice`	PublicGlobalDeviceId
`ParentDeviceManufacturer`	string
`ParentDeviceModel`	string
`PrivateAddType`	int
`RegionalCharset`	string
`SetupState`	int
`State`	int

FIELD	TYPE
`SuggestedDisplay`	string
`Transport`	int

Activity

FIELD	TYPE
`AccountId`	AccountId
`ActivityDisplayName`	string
`ActivityGroup`	ActivityGroup
`ActivityOrder`	short
`Alternatives`	string
`BaseImageUri`	string
`DateCreated`	DateTime
`DateModified`	DateTime
`DefaultChannel`	string
`DefaultStation`	string
`DefaultStationName`	string
`EnterActions`	AbstractActivityAction list
`Icon`	string
`Id`	ActivityId
`ImageKey`	string
`IsDefault`	bool
`IsMultiZone`	bool
`IsTuningDefault`	bool
`LeaveActions`	AbstractActivityAction list
`Name`	string
`Roles`	AbstractActivityRole list
`StartScreen`	string
`State`	ActivityState
`SuggestedDisplay`	string
`Type`	ActivityType

IrProtocol

The vendor's own description of an infrared protocol, and **no capture exercises it**, so this is schema without instances. The service is alive and can be asked, which is the cheapest open lead here.

FIELD	TYPE
`Id`	ProtocolId
`Name`	string
`Attributes`	AttributeType list
`CarrierFrequency`	long
`CodeSegments`	CodeSegment list
`Flags`	FlagType list
`HoldDelay`	long
`HoldMinimumRepeats`	long
`IRSegments`	IRSegment list
`IsFullSequence`	bool
`IsPadded`	bool
`IsPublic`	bool
`KeyCode`	ParsedKeyCode
`NumberOfLinkedLanguage`	long
`PressMinimumRepeats`	long
`Rating`	long
`RelatedProtocols`	ProtocolRelation list
`SendingType`	IRProtocolSendingTypeContract
`Status`	string

What an activity actually does

This section did not exist until 30 August 2026 and its absence is the reason to read the rest sceptically. The `Activity` table above names `EnterActions`, `LeaveActions` and `Roles`, with the types `AbstractActivityAction` and `AbstractActivityRole` beside them, and nothing anywhere said what those were. Danny asked. They turn out to be the part of this model that says what a Harmony **does** when you press an activity, which is the most useful thing in it.

`make model-activity` draws this section: `Activity` with its roles and its actions, which the core diagram cannot show because neither type is a core entity. The role types are drawn as one box listing their names rather than as 22 boxes, for the reason given below, that they carry no fields.

The roles say what each device is for

An activity holds a list of roles, one per device it drives. Every role extends `AbstractActivityRole`, and **the base carries all the substance**: `DeviceId`, `Id`, `NextDevicePowerOnDelay`, `PowerOffOrder`, `PowerOnOrder`, `SelectedInput`.

So one role says: this device, in this position in the power on order and this position in the power off order, waits this long before the next device is switched on, and switches to this input. That is an activity start sequence described in words rather than in bytes.

The 22 role types carry almost nothing of their own. 21 of them declare no field at all, so the class name is the entire value; the exception is `PowerInputActivityRole`, which adds `DeviceClassificationName`. Read it as a vocabulary rather than as 22 structures:

- `AccessInternetActivityRole`
- `ChannelChangingActivityRole`
- `ControlsAppActivityRole`
- `ControlsAppleTVActivityRole`
- `ControlsComputerActivityRole`
- `ControlsMediaPlayerActivityRole`
- `ControlsNetflixActivityRole`
- `ControlsRokuActivityRole`
- `ControlsSonosActivityRole`
- `ControlsSpeakerActivityRole`
- `ControlsVideoCallActivityRole`
- `DisplayActivityRole`
- `KeyboardTextEntryActivityRole`
- `PassThroughActivityRole`

- `PlayGameActivityRole`
- `PlayMediaActivityRole`
- `PlayMovieActivityRole`
- `PowerInputActivityRole`
- `RunLogitechGoogleTVActivityRole`
- `SilentActivityRole`
- `SmartTVActivityRole`
- `VolumeActivityRole`

They correspond to a device's declared capabilities, but only partly, and the gap is not explained. `DeviceCapabilityType` has 81 values and 14 of the 22 role names appear among them exactly. Two more are the same word in the other order or with a prefix, `ChannelChanging` against `ChangingChannel` and `SmartTV` against `IsSmartTV`. The remaining six, `ControlsApp`, `ControlsSpeaker`, `ControlsVideoCall`, `PassThrough`, `PowerInput`, `Silent`, match nothing in the capability list. Whether a device's capabilities decide which roles it may take is **not established**: it is the obvious reading and no capture here exercises it.

The actions are the start sequence, and one of them is a delay

`EnterActions` and `LeaveActions` are ordered lists. Every action extends `AbstractActivityAction`, which carries `ActionOrder`, `Id`, and there are exactly three kinds:

ACTION	WHAT IT ADDS	WHAT IT DOES
`CommandActivityAction`	`CommandName`, `DeviceId`, `TargetLevel`	send a named command to one device
`ChannelActivityAction`	`ChannelNumber`, `DeviceId`	tune one device to a channel
`DelayActivityAction`	`Duration`	wait

A command is named rather than numbered, which is the same distinction section 220 found in the function maps: a configuration numbers a command and the platform names it.

The unit of `Duration` is **not stated in the schema** and nothing here has measured it. Neither is the unit of `NextDevicePowerOnDelay` on a role. Both are open and both are cheap to settle, see the leads at the end.

The button maps, which say which button sends what

A second cluster, 31 types, and none of it was described here before 30 August 2026. The core diagram draws none of it, because nothing in it is reachable from a household: the maps hang off a remote by identifier rather than by reference. That is why it stayed invisible while the account side was documented twice over.

A device map and an activity map are two types, not one

`AbstractButtonMap` carries `ButtonMapId`, `ButtonMapIdentifier`, `ButtonMapSurfaceId`, `Buttons`, `DateModified`, `RemoteId`, `Sequences`, `SurfaceId`, and it has exactly three subclasses:

MAP	WHAT KEYS IT
`DeviceButtonMap`	`DeviceId`
`ActivityButtonMap`	`ActivityId`
`RootButtonMap`	nothing: it adds no field at all

This is Logitech stating the operating concept in their own schema, and it is the first independent source for it. `docs/how-a-harmony-works.md` argues that a device's map and an activity's map are two maps of the same keypad, authored separately, and it was written from their help pages and from measurements over the corpus. Here the two are separate types with a shared base, keyed by the two different things.

`RootButtonMap` **is the interesting one and it bears on an open question.** It extends the base and adds nothing, so it is a map belonging to neither a device nor an activity, which leaves the remote itself. Where device mode's own keypad map lives is open here, section 151: every keypad map in the corpus that sends a code is installed by an activity, and three readings of that remain. A map class keyed by nothing is consistent with the reading that the remote carries a base map, and it is **not** evidence that it does: this is a schema for a later generation of hardware and no capture here holds one. Worth testing, not worth believing.

A button has three actions, not one

`AbstractRemoteButton` carries `ButtonAction`, `ButtonDoublePressAction`, `ButtonId`, `ButtonLongPressAction`, `ButtonState`, `FunctionGroupType`. So a single button holds a **press**, a **long press** and a **double press** action independently. Any editor that models a button as one binding is wrong by a factor of three.

The button kinds are the remote's physical and drawn surfaces:

KIND	ADDS	WHAT IT IS
`HardRemoteButton`	`ButtonKey`	a key on the keypad
`SlideOutKeypadButton`	nothing	a key under the slider, on the models that have one
`KeyboardButton`	`ActiveKeys`, `HasPassThroughSupport`	a key on a keyboard accessory, which can pass keystrokes through
`SoftRemoteButton`	`ButtonImageKey`, `ButtonImagePath`, `ImageId`, `MenuItem`, `TextOnRemote`	a drawn button on the screen, with its image, its label and its place in a menu
`GestureRemoteButton`	`ButtonImageKey`, `ButtonKey`	a gesture on a touch surface
`VoiceRemoteButton`	`ButtonKey`	a voice control

`SoftRemoteButton` **carries a** `MenuItem`, which is a menu name and an index in it. That is the screen page structure this project reads out of a config, named by the vendor.

What a button can be bound to

`AbstractButtonAction` carries `EventType`, `Id`, `Order` and has 8 subclasses:

ACTION	ADDS	WHAT IT DOES
`ButtonActivityAction`	`ActivityId`	start an activity
`ButtonChannelAction`	`ChannelNumber`, `DeviceId`	tune a device to a channel
`ButtonClientAction`	`ActionName`	do something in the client rather than on the remote
`ButtonCommandAction`	`CommandName`, `DeviceId`, `FunctionId`	send one command to one device
`ButtonDelayAction`	`Duration`	wait
`ButtonHomeControlAction`	`ChangeType`, `DeviceId`, `GroupId`, `Property`, `SubDeviceId`, `Targets`, `Value`	change a home automation target
`ButtonProgramAction`	`ProgramId`, `Rule`	run a rule
`ButtonSequenceAction`	`SequenceId`	run a stored sequence, by identifier

`EventType` **on the base is the vendor's word for a distinction this project derived from bytes.** A key code in a configuration is an event type plus a scan code, press, release and repeat, which was got wrong once here by splitting the byte the other way. The platform carries the same idea as a field on every button action. The values are not in the schema.

A sequence is a first class thing: `Sequence` holds `Actions`, `Name`, `SequenceId`, so it is an ordered list of the same actions a button can carry, stored on the map and referenced by identifier. A button map holds its sequences directly, which is what `Sequences` on the base means.

The function maps are the other half, and they are not the same thing

`AbstractFunctionMap` carries `FunctionGroups`, `UIModeName`, and splits the same two ways, `DeviceFunctionMap` and `ActivityFunctionMap`. A function map is a **list of commands offered**, grouped and labelled; a button map is **what each key does**. `FunctionGroup` and `FunctionAction` both extend `FunctionBase`, so a group can contain groups.

`FunctionAction` carries `CommandName`, `DeviceId`, `FunctionId`, `Label`, `TransportType`. `TransportType` **is the field section 220 measured**, where a named group states no transport and the catch all states infrared.

The device catalogue, and what it says about an infrared code

The third cluster, 75 types, and about half of it is search criteria, one class per way of asking the catalogue a question, which says nothing about hardware and is listed in `docs/myharmony/entities.md` rather than described here. What the other half describes is a **code**, and it is worth more to this project than the rest of the model put together, because sections 159 to 171 derived the same structure out of bytes without ever seeing this.

`make model-cluster CLUSTER=catalogue` draws it.

The pieces

`IrProtocol` extends `AbstractProtocol`, alongside `BluetoothProtocol`, `HidProtocol`, `RfProtocol` and `UsbHidProtocol`, so infrared is one transport among five in their model. It carries `Attributes`, `CarrierFrequency`, `CodeSegments`, `Flags`, `HoldDelay`, `HoldMinimumRepeats`, `IRSegments`, `IsFullSequence`, `IsPadded`, `IsPublic`, `KeyCode`, `NumberOfLinkedLanguage`, `PressMinimumRepeats`, `Rating`, `RelatedProtocols`, `SendingType`, `Status`.

TYPE	FIELDS	WHAT IT IS
`CodeSegment`	`Atoms`, `Header`, `Payload`, `TotalLength`, `Trailer`	a segment stated as a code: a header, a payload and a trailer
`IRSegment`	`Header`, `Payload`, `TotalLength`, `Trailer`	a segment stated as timings, with the same three parts
`Payload`	`EncodingType`, `Encodings`, `NumberOfBits`, `ToggleBit`	the bits themselves, their encoding, how many, and a toggle bit
`IREncoding`	`Atoms`, `BitType`	how one digit value is spelled as durations
`Atom`	`MaxValue`, `MinValue`, `Type`, `Value`	one duration, with a nominal value and a tolerance band
`ParsedKeyCode`	`Finish`, `Repeat`, `Start`	which segments make the start, the repeat and the finish
`KeyCodeElement`	`SegmentName`, `SegmentType`	a reference to one segment by name and kind

Four things it confirms that were measured here the hard way

Each of these was derived from configurations, with no access to this schema. They agree.

A code states its frames in two slots. Section 159 found that reading one slot refused every Toshiba code in the catalogue and sent half a command on the families that fill both. Here there are literally two collections, `CodeSegments` and `IRSegments`, and the enumeration that says which one a reference points at has exactly two values: `IRSegment`, `CodeSegment`.

A record has three block pointers: once, held and tail. `ParsedKeyCode` has exactly three fields and they are `Finish`, `Repeat`, `Start`. That is the same three, in the vendor's words.

`Quad` **in a family name is the base of its digits, not a count of its frames.** That cost a day and was settled by a width check that accepted all 69 codes of the family once the base was read as four. The encoding enumeration is `BitEncoding`, `BiphasicEncoding`, `QuadEncoding`, `HexEncoding`, `ByteEncoding`, which is base two, base four, base sixteen and base two hundred and fifty six, plus biphase. The reading was right and this is a second source for it.

A duration is a mark or a space. `AtomType` is `Space`, `Pulse` and nothing else, and an atom carries a tolerance band around its nominal value, which is what makes a stated rhythm matchable against a captured one.

And three things it says that this project has not measured

- `Atom` **has a** `MinValue` **and a** `MaxValue`**.** Every duration this project reads is a single number. Whether the tolerance is stored per code or applied by the compiler is not known here.
- `FlagType` **is** `HighBitRate`, `NoCarrier`, `HighFrequency`, `BitToggle`. `NoCarrier` is a code sent with no carrier at all, which no config in the corpus has been shown to hold, and `BitToggle` pairs with `Payload.ToggleBit`.
- `RelationType` **is** `VerySimilarTo`, `ModeratelySimilarTo`, `PromotedInFavorOf`, `MadeObsoleteFor`, `IsPrototypeOf`, `HasPrototype`, `Overlapping`. So the catalogue records that two protocols are near neighbours, which is exactly the judgement section 162 had to make by hand when their analyser named a Sharp code as `Makita 10 Bit`.

All of this is **client sourced** under decision 2 and none of it has been exercised against a live reply: `Raw` is null on all 5219 commands the census fetched, so the catalogue serves a protocol name and a frame value rather than these structures. What they are good for is knowing what to ask for, and knowing that our reading of a code matches the shape their compiler emits.

The vocabularies

Enumerations are the part that transfers most directly into an interface, because they are the product's own words for a closed set of choices. **All nine that the core reaches are here**; this listed four until 30 August 2026 and said nothing about the other five, two of which are the largest in the model.

- `DeviceCategory`, 6: `Any`, `AudioVideoReceiver`, `SetTopBox`, `Television`, `BluRayDvdPlayer`, `CompansionBox`
- `ActivityType`, 17: `NotSpecified`, `WatchTV`, `WatchDvd`, `PlayGame`, `ListenToMusic`, `Custom`, `SurfWeb`, `WatchNetflix`, `MakeVideoCall`, `WatchAppleTV`, `WatchRoku`, `PCTV`, `SmartTV`, `WatchFireTV`, `ListenToSonos`, `WatchApp`, `ListenToSpeaker`
- `ActivityState`, 2: `Setup`, `NonSetup`
- `ActivityGroup`, 5: `VirtualGeneric`, `VirtualTelevisionN`, `VirtualDvd`, `VirtualCdMulti`, `VirtualGameConsole`
- `KeyboardLayoutType`, 4: `Undefined`, `QWERTY`, `QWERTZ`, `AZERTY`
- `Region`, 5: `Unknown`, `Amr`, `Emea`, `Global`, `Apac`

Three are too long to quote whole and are listed in full in `docs/myharmony/entities.md`:

- `DeviceCapabilityType`, 81 values, what a device declares it can do. It is the list the activity roles above partly correspond to, and it reaches well beyond this era of remote: `ZigBee`, `ZWave`, `Thermostat` and `AlexaSupported` sit in it alongside `Volume` and `ChangingChannel`.
- `DeviceType`, 61 values, and `Icon`, 63. **These are one list, not two.** Every `DeviceType` value is also an `Icon` value, in the same order, and `Icon` adds exactly two that `DeviceType` lacks: `Revue` and `PCTV`. So the picture a device shows and the kind of thing it is are the same vocabulary, which is worth knowing before anyone builds two pickers.
- `CountryType`, 242 values, the ISO country codes plus `XX` and `Unknown`. `Country` pairs one with a name and a `Region`, which is the five way split the product ships against.

Every area, by how many contracts it holds

The area is the last part of the server side namespace each contract declares, so these are the service's own divisions rather than ours. `Data` is the shared bucket, `Logitech.Harmony.Services.Common.Contracts.Data`.

The column sums to 426 rather than to 470, because 44 service contracts declare no namespace at all and so belong to no area. They are counted in the 470 and appear in no row below.

AREA	CONTRACTS
`Data`	163
`Activity`	38
`Search`	37
`UserButtonMapping`	29
`Account`	26
`Infrared`	18
`Operation`	17
`UserFeature`	16
`DataContract`	16
`GlobalLanguage`	10
`ButtonMapping`	9
`FunctionMapping`	6
`Protocol`	5
`Logging`	5
`Release321`	5
`SearchMatch`	4
`Brand`	4
`RemoteInventory`	3
`com/harmony/services/romdata`	3
`Security`	2
`Error`	2
`Discovery`	2
`Service`	1
`Compile`	1
`Services`	1
`Device`	1
`AmazonS3`	1
`RF`	1

The typed identifier family

25 of the types the core reaches are identifiers, and a reader who opens the data sees them as empty. They are not: each extends `AbstractId`, which carries `IsPersisted` and `Value`. So an `AccountId` is a value plus a flag saying whether it has been written down yet, and the only reason there are 25 of them rather than one is that the type stops an account identifier being passed where a remote identifier belongs.

Listed once, because 22 of the 25 hold nothing the others do not. **Three do add a field of their own**, and they are the ones to look at rather than to skim past: `CompilationId` carries a `CompileQueueKey`, `GlobalDeviceVersionId` carries `IsDefault` and `VersionId`, and `GlobalLanguageVersionId` carries `VersionId`. The first of those is the handle a compile is queued under, which is the operation the `myharmony-service` skill drives.

`AbstractAliasId`, `AbstractDeviceId`, `AccountId`, `ActivityId`, `ActivityInputStateId`, `ActivityRoleId`, `AggregatedDeviceId`, `BrandId`, `ButtonImageId`, `ButtonMapId`, `CommandId`, `CompilationId`, `DeviceFeatureId`, `FeatureId`, `FunctionId`, `GlobalDeviceVersionId`, `GlobalLanguageVersionId`, `HouseholdId`, `PairingId`, `PictureId`, `ProductId`, `ProtocolId`, `RemoteId`, `RoomId`, `SurfaceId`.

`AbstractAliasId`, `AbstractDeviceId` and `GlobalDeviceId` sit in the middle of that tree, adding nothing themselves, which is why the count of leaves is larger than the useful vocabulary.

The classes the core reaches, in full

Seven types are reached from the core and are neither an identifier, a vocabulary nor an abstract base. They are small and none of them was described here before 30 August 2026.

TYPE	FIELDS	WHAT IT IS
`ActivityInputState`	`ChannelNumber`, `Id`, `Name`	the input an activity switches a device to, with an optional channel
`AutoDetectedDevice`	`Class`, `Id`, `Manufacturer`, `Model`, `Name`, `USN`	what network discovery found, before a user confirms it is theirs
`ChannelTuningFeature`	`DeviceId`, `FixedDigits`, `Id`	how many digits a device wants when a channel is entered
`Country`	`Code`, `Name`, `Region`	a country code, its name and the region it belongs to
`GlobalDeviceGlobalLanguage`	`GlobalLanguage`, `IsDefault`	a language a catalogue device is available in, and whether it is the default
`GlobalLanguages`	`Id`, `Name`	a language version, by identifier and name
`PrioritizedDeviceCapability`	`DeviceCapability`, `Priority`	one capability a device declares, with a priority against the others

And four abstract bases carry fields that their subclasses rely on:

BASE	EXTENDED BY	FIELDS
`AbstractDevice`	2	`DecodedEdid`, `DefaultInterDeviceDelay`, `DefaultInterKeyDelay`, `DefaultPressMinRepeats`, `DeviceOrder`, `DeviceType`, `EncodedEdid`, `HoldInterDeviceDelay`, `HoldInterKeyDelay`, `HoldMinRepeats`, `InterDeviceDelay`, `InterKeyDelay`, `Manufacturer`, `Model`, `Name`, `PictureId`, `PressMinRepeats`
`AbstractActivityRole`	22	`DeviceId`, `Id`, `NextDevicePowerOnDelay`, `PowerOffOrder`, `PowerOnOrder`, `SelectedInput`
`AbstractActivityAction`	3	`ActionOrder`, `Id`
`AbstractId`	25	`IsPersisted`, `Value`

`AbstractDevice` is the one that had already caused a mistake. `Device` extends it and so inherits 17 fields, `Name`, `Model` and `Manufacturer` among them. The `Device` table in this document listed only the 32 it declares itself, and the test that checked that table read the same field list, so a check named "the field names match exactly" passed over a device with no name in it for five days. Both read every field now, inherited ones included.

Completeness

What "complete" means here, and what it does not. The platform declares 470 service contracts. Strip the enumerations and the identifier wrappers, which deserve no explanation beyond the family they belong to, and 362 carry substance. Those are not one graph: they fall into three sizeable islands and a long tail of request and response types that reference nothing.

CLUSTER	TYPES	DESCRIBED
the account: what a household holds	121	the twelve the core diagram draws, plus the activity's roles and actions
the device catalogue	75	the half that describes a code; the search criteria are listed, not described
button and function maps	31	all of it

150 types are reachable from the roots those clusters start at, following both fields and inheritance. Every one of them is listed with all of its fields in `docs/myharmony/entities.md`, and a test fails if that stops being true.

Three honest limits. The rail walks from named roots, so a fourth subsystem connected to none of them would sit outside it exactly as the button maps did until 30 August 2026; adding a root is what brings one in. The tail of request and response types is listed and not described, because a class whose whole content is one identifier and a flag says nothing a reader needs. And **listed is not explained**: the reference states every field of every contract, while the prose above covers the three clusters and not the tail.

Before 30 August 2026 the figure was twelve explained out of 470, with the rest named only as a type in a field table. Danny found that from two directions, first by asking what `AbstractActivityAction` was and then by observing that the core drawing was very small for the size of the model. Both were right and the second was the more useful, because it pointed at whole subsystems rather than at one gap.

What can be asked of it

The entity half above says what the platform **holds**. This says what it can be **asked**, which is the other thing a schema is good for and the part an importer needs. `docs/myharmony/operations.json` is the data.

The proxy declares each operation twice, once as the request carrying the wire action and its typed parameters and once as the reply, and joining the two gives 298 operations over 19 service interfaces with a reply type resolved for every one of them.

SERVICE INTERFACE	OPERATIONS
`UserAccountDirector.IUserAccountDirector`	63
`UserButtonMappingManager.IUserButtonMappingManager`	36
`DeviceManager.IDeviceManager`	33
`AccountManager.IAccountManager`	32
`SecurityDirector.ISecurity`	21
`ActivityManager.IActivityManager`	20
`RemoteManager.IRemoteManager`	18
`AmazonS3ImageManager.AmazonS3`	16
`ProductManager.IProductManager`	10
`SecurityDirector.ILinkSecurity`	9
`CompileManager.ICompileManager`	8
`UserFeatureManager.IUserFeatureManager`	8
`DiscoveryService.IDiscovery`	5
`GlobalDeviceManager.IGlobalDeviceManager`	5
`Authentication.AuthenticationService`	4
`DeletionManager.IDeletionManager`	4
`InfraredAnalysisManager.IInfraredAnalysisManager`	3
`RomDataService.IRomDataLibrary`	2
`HelpContent.IHelpContentService`	1

Three clients and one service, and the three do not agree, which is the reason to keep them apart rather than quote one number. Harmony Desktop's web application declares 78 operations over 14 services; this proxy declares 298 over 19; and the live service's own Discovery listing advertises 308 over 50.

Eleven services appear in both this proxy and the Discovery listing. The proxy names operations the listing does not on **all eleven**, the listing returns the favour on **seven**, and on those seven each source is missing something the other has. So neither is a superset of the other, the platform is larger than any single count of it, and the four remaining services are the only ones where one source covers the other. The largest disagreement is `UserAccountDirector`, 49 advertised against 63 declared with 26 in common.

That paragraph first said "on every one of those eleven", generalised from a sample of three services, and the test written for it refuted it within a minute on four.

Seven services the proxy declares are absent from the Discovery listing altogether, and they are the ones a client reaches at a fixed address rather than by discovery: `Discovery` itself, both security interfaces, products, help content, the ROM data library, the Amazon image service and ASP.NET's own authentication service.

Which operation can hand back which entity

Computed through inheritance, because a reply typed as a base class reaches everything below it. Reading the reply type literally is wrong in exactly the case that matters: it says nothing returns an `IrProtocol`, when `GetProtocolList` returns a list of that type's base class. 233 of the model's types are reachable that way.

ENTITY	OPERATIONS THAT CAN RETURN ONE
Activity	16
Account	9
Device	7
Remote	7
`IrProtocol`	**1**

That one operation is broken on the live service. `UserAccountDirector/ProtocolList` is advertised, answers a POST with 405 and a GET with a bare gateway error, reproducibly, on two accounts. Two neighbours on the same account scoped address answer normally, `ActivityList` and `FunctionList`, so the address form and the verb are right and the failure belongs to those endpoints: `DeviceList` and `CapabilityList` fail the same way. So `IrProtocol` stays schema without instances, and the reason is now measured rather than unknown.

The verb is worth writing down, because it is not the one everything else here uses: the account scoped addresses are `.../json/Account/<account>/<Name>` and they want a **GET**. Every other call on this platform is a POST of a JSON body, and a POST to these answers 405.

What a device's commands are called

A configuration addresses infrared codes **by number**. Nothing in it says which code is volume up. The platform holds that layer separately, as a **function map**: one per device and one per activity, each a set of named groups of functions, where a function carries a command name, a label for a person to read, and an identifier.

Both kinds of map are the same shape in the schema, `DeviceFunctionMap` and `ActivityFunctionMap` extending one `AbstractFunctionMap` that holds the groups. That is the operating concept in `docs/how-a-harmony-works.md` stated by the vendor: a device's map and an activity's map are two maps of the same keypad. A device map names the default mode, an activity map names its own activity.

A named group against the catch all beside it

A function in a named group states no transport; a function in `Miscellaneous` states infrared. No exceptions in the two captures of 30 August 2026, 1191 functions. So the named groups are the platform's button vocabulary, held independently of how a command reaches the device, and `Miscellaneous` is where a device's own commands live, which exist only as a concrete infrared code. `TransportType` has nine values including HDMI, three flavours of Bluetooth and two of network, so the transport independence is real and not an artefact of an era that only had infrared.

GROUP	COMMAND NAMES
`Channel`	`ChannelDown`, `ChannelPrev`, `ChannelUp`, `Prev`, `Previous`
`ColoredButtons`	`Blue`, `Green`, `Red`, `Yellow`
`DisplayMode`	`Aspect`, `Display`
`GameType2`	`Circle`, `Cross`, `Square`, `Triangle`
`GameType3`	`Enter`, `Home`, `OK`, `Ok`, `Select`
`GoogleTVNavigation`	`Netflix`, `Settings`
`MediaCenter`	`MyMusic`, `MyPictures`, `MyTv`, `MyVideos`
`NavigationBasic`	`DirectionDown`, `DirectionLeft`, `DirectionRight`, `DirectionUp`, `Enter`, `OK`, `Ok`, `Select`
`NavigationDSTB`	`Favorite`, `List`, `Live`, `RecordedTV`, `Search`
`NavigationDVD`	`Angle`, `Audio`, `Back`, `Menu`, `PopUpMenu`, `Return`, `Subtitle`, `TopMenu`
`NavigationExtended`	`Cancel`, `Clear`, `Exit`, `Guide`, `Info`, `Options`, `PS`, `PageDown`, `PageUp`, `StepBack`, `StepForward`, `TopMenu`
`NumericBasic`	`#`, `*`, `-`, `.`, `0`, `1`, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`, `Clear`, `Enter`
`NumericExtended`	`+10`
`PictureAdjustment`	`PictureMode`
`PictureInPicture`	`PIP`
`Power`	`PowerOff`, `PowerOn`, `PowerToggle`
`RadioTuner`	`PresetNext`, `PresetPrev`, `TuneDown`, `TuneUp`
`Setup`	`Setup`, `Sleep`
`SoundModes`	`Effect`, `SoundMode`
`Teletext`	`Teletext`
`TransportBasic`	`Eject`, `FastForward`, `Pause`, `Play`, `QuickStop`, `Rewind`, `Stop`, `iPodFastForward`, `iPodPause`, `iPodPlay`, `iPodRewind`, `iPodStop`
`TransportExtended`	`Next`, `NextTrack`, `Prev`, `PreviousTrack`, `SkipBack`, `SkipBackward`, `SkipForward`
`TransportRecording`	`Record`
`Volume`	`Mute`, `VolumeDown`, `VolumeUp`

This table is a sample and it is a floor, not the platform's vocabulary. It is the union of what the devices on **two test accounts** happened to use on **30 August 2026**, so a group the platform has and those devices did not use is simply absent from it. Those accounts are working accounts whose contents change as experiments need, so nothing here should be read as a statement about what an account contains, and every count below is a count of a dated capture.

It is also not an enumeration and must not be treated as one. It carries `OK` beside `Ok`, `Prev` beside `Previous`, `SkipBack` beside `SkipBackward`, and a set of `iPod` prefixed transport commands, so it is the union of what manufacturers' own command sets are called, bucketed into groups, rather than a closed list somebody curated. Ten command names sit in more than one group, so a group is not a function of a name either.

Those two captures give 105 command names in named groups and 301 device specific ones. The device specific names are Logitech's own database content for particular devices and stay in the lab; the groups and the names above are vocabulary.

The two sets overlap, on `+10`, `Home`, `Options`, `PS`, `PresetNext`, `PresetPrev`, `Select` and `Stop`. So the platform files the same command name in a named group for one device and in the catch all for another, and being device specific is a property of a device's entry rather than of a name.

What bears on work already in flight

- `DefaultInterKeyDelay`, `DefaultInterDeviceDelay` **and** `DefaultPressMinRepeats` are in the schema, on `AbstractDevice`, which `Device` extends. **This said they were in live replies and "not in the schema"** until 30 August 2026, which was wrong for the dull reason that the reading looked at a device's own field list and not at what it inherits. Nine timing fields sit on that base: `DefaultInterKeyDelay`, `InterKeyDelay`, `HoldInterKeyDelay`, `DefaultInterDeviceDelay`, `InterDeviceDelay`, `HoldInterDeviceDelay`, `DefaultPressMinRepeats`, `PressMinRepeats` and `HoldMinRepeats`. `docs/predictions-sequence-delay.md` is about exactly these quantities, scored against a config; here the platform names them, and it distinguishes a default from a current value and a press from a hold, which that prediction did not.
- `DelayActivityAction.Duration` **and** `AbstractActivityRole.NextDevicePowerOnDelay` are the two delays an activity states, and **neither unit is given**. The corpus states its sequence delays in tenths of a second. Comparing the two is a corroboration by routes with nothing in common, and it needs one captured activity with a delay in it rather than any new protocol work.
- `IsInterKeyDelayOptimized` is in the schema, so the delay is something the platform tunes rather than a constant.
- `IrProtocol` **carries two segment collections**, `CodeSegments` and `IRSegments`, which is section 159's finding that a code states its frames in two slots, in the vendor's own vocabulary.
- `GlobalRemoteSkinId` **sits beside** `SkinId` on a remote, so the trap the `myharmony-service` skill documents is two genuine fields rather than one misread.
- `DefaultChannel`, `DefaultStation` **and** `DefaultStationName` sit on an activity, next to the favourite channel work of sections 154 and 156.
- `DeviceCategory` **has six values** where the hub generation's icon set had 95 names, `reference/device-categories.md`. The categories a Harmony One era account can hold are the six.